Image Reconstruction

COMP0210 2024/25: Research Computing with C++ Assignment 1

Contents

Errata . . . 2
Late Submission Policy . . . 2
Extenuating Circumstances . . . 2
Collaboration and ChatGPT / AI Tools . . . 2
Assignment Starter Download and Submission . . . 3
Naming and Build Structure . . . 4
Installing FFTW3 . . . 5
Reporting Errors . . . 5

Assignment: Image Reconstruction with Optimisation Algorithms 5
0.1 Mathematical Background . . . 5
0.1.1 The Optimisation Problem . . . 5
0.1.2 An Example Problem . . . 6
0.1.3 Solving the Optimisation Problem: Iteration and Convergence . . . 9
0.1.4 The Initial Guess . . . 10
0.2 Third Party Libraries: The FFTW Library . . . 10
0.2.1 Fourier Transforms . . . 10
0.2.2 Discrete Cosine Transforms . . . 11
0.2.3 Data order and $\vec{k}$ values . . . 11
0.2.4 Scaling in the FFTW Library . . . 11
0.2.5 FFTW input / output buffers . . . 11
0.2.6 FFTW plans and executing the Fourier transform . . . 12
0.3 The Initial Repository . . . 14
0.4 Writing In `Responses.md` . . . 15

1 Library Code Part 1: I/O [8 Marks] 15

2 Library Code Part 2: The Generic Problem [12 marks] 15
2.1 Abstract Classes [4 marks] . . . 15
2.1.1 Differentiable Functions . . . 16
2.1.2 Non-Differentiable Functions . . . 16
2.1.3 Linear Operators . . . 16
2.2 The Iterative Algorithm [5 marks] . . . 17
2.3 A Simple Test [3 marks] . . . 17

3 Library Code Part 2: Concrete Linear Operators [21 marks] 18
3.1 Implementing a Sub-Sampling Linear Operator [7 marks] . . . 18
3.2 Implementing a Convolution Linear Operator [14 marks] . . . 19

4 Library Code Part 4: Concrete Functions [22 marks] 20
4.1 Implementing a Gaussian Likelihood function [6 marks] 20
4.2 Implementing an DCT L1-Norm Prior [9 marks] 21
4.3 Convergence Functions [3 marks] 22
4.4 Integration Test [4 marks] 22

5 App Code Part 1: Simple Imaging Problems [10 marks] 23

6 Library Code Part 4: Further Operators and Abstract Operator Composition [12 marks] 24
6.1 Fourier Transform Operator [3 marks] 25
6.2 Operator Composition [9 marks] 25

7 App Code Part 2: A More Complex Imaging Problem [5 marks] 26

8 Additional Considerations (10 marks) 26
8.1 Good practices 26
8.2 User instructions 27

9 Additional Hints 27
9.1 Code Layout 27
9.2 Unit Tests 27
9.3 Commit Log 28

Errata

If you believe you have found an error in this assignment, please contact the module leader as soon as possible.

- Corrected signs in description of algorithm in section 0.1.3; you should subtract the changes to the image vector rather than add them.
- Correct test image named to `TestSphere.pgm` in section 4.4
- Corrected order of axes in FFTW library calls

Late Submission Policy

Please be aware of the UCL Late Submission Policy (Section 3.12).

Extenuating Circumstances

The course tutors are not allowed to grant Extenuating Circumstances for late submission. You must apply by submitting an Extenuating Circumstances form which can be found on the Extenuating Circumstances information website. The Department EC Panel will then decide whether or not to grant extenuating circumstances and email the module lead confirming that Extenuating Circumstances have been granted and providing a revised submission date.

Collaboration and ChatGPT / AI Tools

This assignment must be you own work. You are not permitted to share or collaborate on code, nor are you permitted to use AI tools such as ChatGPT or github co-pilot to produce any code or question responses.

You may discuss the background of the assignment to help you understand the problem at hand, but not any solutions.

You are strongly encouraged to use the Moodle forums as a means of asking and responding to questions. You can also contact the lecturer directly via email or booking office hours.

Assignment Starter Download and Submission

The starter repository must be downloaded from the github link below, which will generate your personal github repository. You can use GitHub for your work if you wish, although we will only mark the code you submit on Moodle. Due to the need to avoid plagiarism, **do not use a public GitHub repository for your work**. We have provided a private repo unique to you accessible through Github Classroom. **To access the starting project, using the following link**:

https://classroom.github.com/a/LqDHSaBT.

After you accept the permissions, you will have access to a repository named

```
image-reconstruction-<gh_username>
```

for your particular github username. This will contain the starting project for your own work.

Important: It is vital you use the above link to access the starting project.

For this coursework assignment, you must submit by uploading a single Zip format archive to Moodle. You must use only the zip tool, not any other archiver, such as `.tgz` or `.rar`. Inside your zip archive, you must have a single top-level folder, whose folder name is your student number, so that on running unzip this folder appears. This top-level folder must contain all the parts of your solution. Inside the top-level folder should be a git repository named exactly `COMP0210Assignment1`. All code, images, results and documentation should be inside this git repository. You must include the `.git` folder otherwise we cannot mark your git commit log. Your folder structure should look similar to (but not exactly the same as):

```
- 314159   // YOUR STUDENT NUMBER
  - COMP0210Assignment1
    - .git
    - README.md
    - source
    - test
    - ...
```

- **Please remove the `build` folder before zipping and submitting.**
- In order to have this folder structure I recommend starting by creating a folder with your student number, and then moving into that folder to clone the repository.

Important for cloning this repository: - You should clone this repository into a folder called `COMP0210Assignment1` rather than the default name. This is so that everyone has the same folder name and the automated tools can access the folder and build your code.

You should therefore clone with the command:

```
git clone <your private github link> COMP0210Assignment1
```

N.B. We have provided a `.devcontainer` folder in this repository. To develop in the container, make sure that you have Docker running and then select "Reopen in Container" with the top level folder `COMP0210Assignment1` open in VSCode.

You should test that your zip file produces this structure; if you are working on Windows, you should zip/unzip the files **inside WSL**, as the native Windows tool will produce a different folder structure when unzipped. Be sure to test your submission by unzipping the `.zip` file and checking the folder structure **before you submit**; will guarantee that your zip file will extract correctly during the submission process. You can use the command line tool `zip` in WSL / Linux terminals.

You will notice that there are multiple source folders, `Optimisation` (library code), `app` (application code), and `test` (testing code). You should take care to choose an appropriate location for each piece of code you write. **You can, and should, add new source and header files where appropriate. You should consider where they fit into the overall structure of the repository and abide by good practices regarding the grouping of related code in files, and the splitting of declarations (header files) and implementations (source files).**

If writing tests involving paths (e.g. when testing loading a grid from a specific file) **the test executables will be run from the top level directory**. You may hard-code *relative* paths, provided they execute correctly when run from the top level directory.

We recommend you use Visual Studio Code IDE with either Ubuntu, WSL running Ubuntu, or a devcontainer running Ubuntu 22.04 docker image for development (the correct Dockerfile is included for you in the repository). **The devcontainer is recommended for Mac users to minimise compatibility issues.** This is not a requirement but if you do use a different setup then you need to ensure that your code compiles and runs on Ubuntu 20.04 or 22.04, with g++ (enforcing C++17) and CMake 3.21 or above as this is the environment the markers will use to test the code. Your code should not rely on any external dependencies other than Catch2 for testing, and FFTW3 for the Fourier transforms.

This assignment requires installation of the Catch2 and FFTW3 libraries. I also strongly recommend having Valgrind or a similar tool available. All of these are already installed in the devcontainer, so consider using Docker with the container if you have any issues or want to avoid installs on your own machine.

Marks will be deducted for code that does not compile or run. Marks will also be deducted if code is poorly formatted. You can choose to follow your own formatting style but it must be applied consistently. See the Google C++ style guide for a good set of conventions regarding code formatting.

Naming and Build Structure

1. It is important to use the exact given names for any executable files *and* to use the correct command line interface described in the assignment, as these will be compiled and run automatically. You are free to add `.cpp` and `.h` files throughout the project, and name them yourself, but **the executables must retain their names and interfaces**.
2. All of your tests should be compiled into a single test executable, with the given name `TestOptimisation`, as in the initial CMake files. You may use as many additional `.cpp` files as you like, but they should still all compile down to a single executable with this name.
3. All executables **must** be placed in `build/bin`, as in the initial CMake files.
4. Where names of functions or classes etc. are explicitly specified you must use the names given. However, where not specified you are free to choose you own *clear* names for

variables, functions, classes, files, and so on. The same applies to types and function signatures.

Your code **must** be able to be built from the top level folder (`COMP0210Assignment1`) using the following commands:

```
cmake -B build
cmake --build build
```

You can assume that when we build your project we will have the necessary dependencies (FFTW3, Catch2) installed. It is important that you **do not add further dependencies**.

Installing FFTW3

If you are not using Docker you will need to install FFTW3 on your machine. To do so:

1. Download FFTW 3.3.10 from the FFTW website.

- You can also use the terminal command: `wget http://fftw.org/fftw-3.3.10.tar.gz`
- Unzip it with: `tar -xvzf fftw-3.3.10.tar.gz`
- Move into the folder: `cd fftw-3.3.10`
- Build and install:
  - `mkdir build`
  - `cd build`
  - `cmake ..`
  - `make -j`
  - `make install`

You must follow these instructions to build and install using CMake, otherwise CMake will not be able to find FFTW3 when you try to build your project.

Reporting Errors

Contact the lecturer directly if you believe that there is a problem with the starting code, or the build process in general. To make error reporting useful, you should include in your description of the error: error messages, operating system version, compiler version, and an exact sequence of steps to reproduce the error. Questions regarding the clarity of the instructions are allowed. Any corrections or clarifications made after the initial release of this coursework will be written in the errata section and announced.

Assignment: Image Reconstruction with Optimisation Algorithms

In this assignment you will write a library for solving optimisation problems of a given form. This library will be designed to be highly generic, and make use of the design features that we have discussed in C++ and OOP. You will then write application code which uses this library to solve image reconstruction problems from simulations of lossy image data.

0.1 Mathematical Background

0.1.1 The Optimisation Problem

Optimisation problems generally involve finding inputs which minimise or maximise a given function, called a "cost function", of those inputs. Many simple optimisation algorithms involve

using the gradient of the function at a given point as a way of generating a new guess for a point closer to the minimum. In our case, we want to minimise a cost function of this form:

$C(\mathbf{x}) = f(\mathbf{x}, \mathbf{y}, \Phi) + g(\mathbf{x})$

where:

- $\mathbf{x}$ is the input that we want to vary in order to minimise C. In our case, this will be an *image model*, which we are trying to get as close as possible to the "true" image.
- $\mathbf{y}$ is a set of measurements. These are typically noisy, incomplete, and/or distorted in some way. For example, an image may be blurry or have data missing, or suffer from other camera effects that distort the image that it is trying to capture. Our goal is to reverse these processes as well as possible.
- Φ is a linear operator which translates an image model ($\mathbf{x}$) into the measurements ($\mathbf{y}$) that would be made from that image: $\mathbf{y} = \Phi(\mathbf{x})$. (In this assignment we will ignore any random measurement noise, although in practice this is usually present!) Φ is sometimes known as a measurement operator, and can capture things like incompleteness of the image (patches missing) or camera effects such as point spread. For a given linear operator Φ, we will also need its *adjoint* $\Phi^\dagger$. The adjoint is not the same as the inverse, but it is often used as a pseudo-inverse (approximate inverse) for operators which are not invertible. Most linear operators that we are interested in are not invertible.
- f is a differentiable function, which is to say that the gradient ∇f is defined. A commonly use differentiable function in cost functions is a Gaussian log-likelihood $L \sim |\Phi(\mathbf{x}) - \mathbf{y}|^2/2\sigma^2$. (You will not need to be able to differentiate the functions yourselves; we will provide the mathematical functions and their gradients.)
- g is a non-differentiable function, so ∇g is **not** defined, and hence we cannot rely on the gradient alone. Instead we can defined other ways of taking a step motion such as a proximal operator. In our case we will use this function to define a non-smooth prior, commonly found in optimisation problems.[1]

N.B. You do not need to understand the mathematics of gradients, proximals, or linear operators to implement the code, all you need to know is that differentiable and non-differentiable functions are treated separately with one defining a gradient and the other defining a proximal. The functions, gradients, and proximals will be given to you. You should however bear in mind what the *input* and *output* types of each of these kinds of functions are.

This optimisation problem is very general, and not limited to the application to imaging problems that we will look at, nor do f and g need to be a likelihood and prior respectively. To implement this generality we will use abstract and generic code, with increasingly specialised implementations as we work towards solving specific imaging problems.

0.1.2 An Example Problem

Let's consider that the real image that we are trying to recover is this:

[1] A *prior* is a function which encodes our expectations about how an image should look; formally it gives us the probability of an image $\mathbf{x}$ in the absence of any data. Priors often encode features like smoothness or simplicity.

The perfect image is however always unknown to us in real situations. Generally our measurements have imperfect information; let's assume in this case that some of the data is missing. In this case the linear operator Φ would take an image $\mathbf{x}$ and return a *subset* of those pixels. This linear operator *loses information*, and so can't be reversed perfectly without knowing the original image. This means that the *adjoint* of our linear operator cannot reconstruct the image, and instead just places zeroes (black) in the missing pixels. The image that we start with from our measurements is therefore something like this:

From our own intuitive understanding of images we know that this these black pixels everywhere are very unlikely to be correct: these pixels should probably have similar values to pixels that neighbour them. Therefore, using a prior which encourages some kind of smoothness, we can reconstruct a better approximation of the image like the one below:

This image has been reconstructed from the sub-sampled image using the algorithm described in this assignment. The cost function used to reconstruct this image needs to balance the smoothness prior with the fidelity to the data (the pixels that we know are correct shouldn't change too much) to avoid over-smoothing.

0.1.3 Solving the Optimisation Problem: Iteration and Convergence

The solution to the optimisation problem will be a simple iterative algorithm. (This can be made faster and more robust by a number of improvements which are beyond the scope of this assignment!)

The algorithm will stop once either:

- a convergence criterion is met, or
- a maximum number of iterations is reached (to prevent infinite loops).

In each iteration the algorithm will take the estimate $\mathbf{x}_n$ and produce a new estimate $\mathbf{x}_{n+1}$. These are related by the formulae:

$\mathbf{x}' = \mathbf{x_n} - \alpha \nabla f(\mathbf{x}_n)$

$\mathbf{x}_{n+1} = \mathbf{x}' - \beta \text{prox}_g(\mathbf{x}')$

where

- $\mathbf{x}'$ is a temporary intermediate value calculated by taking the gradient step
- prox_g is the proximal operator of the function g
- α and β are constants which control how big the step taken is relative to the gradient / proximal step. Tweaking these parameters can have an impact on how dominant either

function (f or g) is in determining the solution.

The convergence criteria are used to determine when to stop the iterative algorithm. We will explore two common convergence criteria:

- $\frac{|\mathbf{x}_{n+1}-\mathbf{x}|}{|\mathbf{x}|} < \delta$
- $\frac{|C(\mathbf{x}_{n+1})-C(\mathbf{x})|}{C(\mathbf{x})} < \delta$

This means that we are checking that the magnitude of fractional change in either $\mathbf{x}$ itself or the cost function $C(\mathbf{x})$ in a given iteration is smaller than some tolerance δ.

These criteria will be expressed in the form of a function $H(\mathbf{x}_n, \mathbf{x}_{n-1}, \delta)$ which returns True if the convergence criteria are met (stop the algorithm) and False otherwise.

0.1.4 The Initial Guess

The iterations need to start with some initial guess, $\mathbf{x}_0$. This initial image is called the *dirty image*, and is found by making to make an attempt to reverse the effects of the measurement operator, i.e. applying the adjoint of our linear operator to the measurements $\mathbf{y}$:

$\mathbf{x}_0 = \Phi^\dagger(\mathbf{y})$.

If the measurement operator were perfectly invertible, then we would not necessarily need to do anything else![2] Most measurements however involve a loss of information, which cannot be reversed analytically. In the example problem given above many pixels are missing, which cannot be reversed analytically as the sub-sampled image contains *less information* than the true image we are trying to reconstruct.

0.2 Third Party Libraries: The FFTW Library

This project will make use of the very common FFTW Library. The documentation for this library can be confusing, so we will explain how to use the relevant parts of the library. The most important thing that we need to understand is how data is formatted going into and out of these functions, and how data gets scaled by these functions.

We will describe the use of the FFTW library only as it applies to the specific functions and data structures that we use in this assignment. These statements should not be taken as fully general and may not hold for other methods in the FFTW library which have different properties!

All of the following types and methods are available in the `fftw3.h` header file.

0.2.1 Fourier Transforms

The information contained in an image can be represented as a vector of values (one for each pixel), where each value represents the intensity at a particular point in space. More generally, we can represent a function in space in a discretised way by representing it as a vector of values which are the amplitude of the function at regular points in space. Functions in space can also be represented as the *sum of waves with amplitudes at different frequencies*. This is sometimes called the Fourier representation of the function. The Fourier transform (FT) is used to convert

[2] Measurements may also include *random noise*, which we will ignore in this particular assignment. If you're interested in more practical applications of this technique, then some amount of noise would apply to any real-life measurements, even if there were no other distorting effects. So, even if our measurement operator is invertible, you can use the optimisation methods we discuss as a method of *noise reduction* on your image, by applying a prior that enforces smoothness.

a vector of complex numbers representing values in space to another vector of complex numbers representing amplitudes of waves of different frequencies.

0.2.2 Discrete Cosine Transforms

A related transform which is extremely common in image process is the *Discrete Cosine Transform* (DCT). The DCT transforms a buffer of real numbers to a buffer of real numbers. The 2D DCT can be used to describe an image as a series of 2D cosine functions. This avoids dealing with the complex number of the FT. This representation is useful in image processing because viewing the image as combinations of oscillating functions of different frequencies provides information about the structure of the image on different scales.

0.2.3 Data order and $\vec{k}$ values

FFTW can perform multi-dimensional Fourier transforms, but the data buffer is always provided as a single **one-dimensional array**, which means we must adopt some ordering for multi-dimensional data.

For a 2D image with width and height w and h, can convert the 2D row-major indices (i, j) to a flat index μ using the following expression:

$\mu = j + wi.$

This data ordering is used for both input and output buffers.

0.2.4 Scaling in the FFTW Library

The FT and DCT computed by FFTW **are not normalised**. For a normalised FT we would have:

$\mathrm{FT}^{-1}(\mathrm{FT}(\mathbf{x})) = \mathbf{x},$

and likewise for the DCT. However, using the fast Fourier transform as defined in FFTW, we have the following relation instead:

- $\mathrm{FT}^{-1}_{\mathrm{FFTW}}(\mathrm{FT}_{\mathrm{FFTW}}(\mathbf{x})) = N\mathbf{x},$
- $\mathrm{DCT}^{-1}_{\mathrm{FFTW}}(\mathrm{DCT}_{\mathrm{FFTW}}(\mathbf{x})) = 4N\mathbf{x},$

for a **2D transform**, where $N = w \times h$ is the total number of data points (pixels) in the image. Since our method involves a forward and inverse transform, we pick up this extra factor of N for a FT and $4N$ for a DCT along the way, which requires the elements in the buffer to be rescaled by a factor of $\frac{1}{N}$ or $\frac{1}{4N}$ respectively. In order to be consistent with data that you will be provided, **you should use the standard forward transform from the FFTW library and apply the re-scaling after the inverse transform only**. It is important that you implement it this way as some problems that you will be given will require you to read frequency space data that has been provided for you, and this data has been generated from an image using the standard forward transform.

0.2.5 FFTW input / output buffers

The FFT method that we will use takes arrays with elements of the type `fftw_complex`, which is included in the `<fftw3.h>` header. **This is an alias for the type `double[2]`**, which is **not** the same as `std::complex<double>`, so we can access the real and imaginary parts using code such as the following:

```
#include <fftw3.h>
#include <iostream>

int main()
{
  fftw_complex z;

  //set real part
  z[0] = 1.5;

  //set imaginary part:
  z[1] = 0.2;

  //print complex number
  std::cout << z[0] << " + " << z[1] << "i" << std::endl;

  return 0;
}
```

FFTW has its own methods for allocating a buffer in memory, which should be used whenever using the library.

The following line allocates an array called `buffer` to contain `N_elements` values of type `fftw_complex`. The type of `buffer` is `fftw_complex *`, which is a pointer i.e. it is implemented as a C-style array.

```
fftw_complex * buffer = (fftw_complex *) fftw_malloc(sizeof(fftw_complex) * N_elements);
```

Note that this does not *initialise* the elements in the buffer, only allocates memory on the heap for the buffer.

- The FT has separate input and output buffers, and both must be allocated before the FT is called.
- Whatever is in the output buffer will be rewritten with the result of the FT.
- The input and output buffers are the same size.

Buffers which are allocated with `fftw_malloc` must also be freed using `fftw_free`, for example:

```
fftw_complex * buffer = (fftw_complex *) fftw_malloc(sizeof(fftw_complex) * N_elements);

fftw_free(buffer);
```

0.2.6 FFTW plans and executing the Fourier transform

FFTW uses "plans" to describe what transform needs to be performed. Plans determine how many dimensions the data has (ours is two-dimensional), what the input and output buffers will be, and whether you are transforming *to* the frequency representation (the "forward" transform), or *from* the frequency representation (the inverse or "backward" transform)[3].

[3] The mathematical Fourier transform is *unitary*, meaning that the inverse is the same as the adjoint. You will typically therefore see it referred to as the inverse and notated as such, which we will also do in this assignment. However, you should bear in mind that if you define the Fourier transform as a linear operator $\Phi(x) = \mathrm{FT}(x)$, then the adjoint of that operator will be the inverse Fourier transform $\Phi^\dagger(x) = \mathrm{FT}^{-1}(x)$. The FFTW implementation of the FT or DCT is only unitary once it has been renormalised as discussed in the previous section.

The plans do not need to be re-made every time you want to perform the transform: **you can re-use a plan as many times as you like as long as these properties don't change**.

The plans have type `fftw_plan`, and are declared, allocated, and deallocated as follows:

Declaration and Allocation:

The forward plan for the FT is initialised using the `fftw_plan_dft_2d` function with the following arguments:

```
fftw_plan forward_plan = fftw_plan_dft_2d(h,
                                          w,
                                          input_buffer,
                                          output_buffer,
                                          FFTW_FORWARD,
                                          FFTW_MEASURE);
```

- `w` and `h` are the width and height of the image.
- `input_buffer` contains the image information to which you want to apply the transform. This is a `fftw_complex *` type array for the FT.

- `output_buffer` is where you stored the result of the transform. This is also a `fftw_complex *` type array.
- `FFTW_FORWARD` and `FFTW_MEASURE` are constants defined in `fftw3.h`. `FFTW_FORWARD` means that this plan will be used for forward transforms. `FFTW_MEASURE` is a flag which tells FFTW to take some measurements to improve performance. You should keep these as they are!

The inverse plan is initialised similarly:

```
fftw_plan inverse_plan = fftw_plan_dft_2d(h,
                                          w,
                                          input_buffer,
                                          output_buffer,
                                          FFTW_BACKWARD,
                                          FFTW_MEASURE);
```

- Note that `FFTW_FORWARD` has been changed to `FFTW_BACKWARD` to indicate a transform from frequency space to real space.
- The input buffer should be a frequency space representation.
- The output buffer is a real space representation.

The forward and inverse plans for the DCT are declared similarly:

```
fftw_plan forward_plan = fftw_plan_r2r_2d(h,
                                          w,
                                          input_buffer,
                                          output_buffer,
                                          FFTW_REDFT10,
                                          FFTW_REDFT10,
                                          FFTW_MEASURE);
```

- `w` and `h` are the width and height of the image.
- `input_buffer` contains the image information to which you want to apply the transform. This is a `double *` type array for the DCT.

- `output_buffer` is where you stored the result of the transform. This is also a `double *` type array.
- `FFTW_REDFT10` and `FFTW_MEASURE` are constants defined in `fftw3.h`. The first two arguments (`FFTW_REDFT10`) tells FFTW what kind of DCT to apply to each dimension; this one is typically the "standard" DCT. You shouldn't change any of these flags!

```
fftw_plan inverse_plan = fftw_plan_r2r_2d(h,
                                          w,
                                          input_buffer,
                                          output_buffer,
                                          FFTW_REDFT01,
                                          FFTW_REDFT01
                                          FFTW_MEASURE);
```

- Note that `FFTW_REDFT10` has been changed to `FFTW_REDFT01`; this is the "standard" inverse DCT.
- The input buffer should be a frequency space representation.
- The output buffer is a real space representation.

Deallocation:

All plans, regardless of their type, need to be deallocated using the `fftw_destroy_plan` function:

```
fftw_destroy_plan(forward_plan);
fftw_destroy_plan(inverse_plan);
```

0.3 The Initial Repository

Start by familiarising yourself with the starter repository.

There are multiple folders which contain source files:

1. `app` is for application code. These are linked to the `OptimisationLib`.
2. `Optimisation` contains library code such as I/O, algorithms, functions / linear operators etc.
3. `test` for test code. There are subfolders for data and images that will be used during testing.
4. `data` contains data that you will use when running your final applications.
5. `images` is where you will place the image files from your reconstruction process.

You will also have a `README.md` and `Responses.md` file. The `README.md` should be filled in as described in the final section of this assignment, and should contain all the necessary information for a reader to build and run your application. `Responses.md` is used for answering written questions that appear in this assignment, such as discussing your design choices.

You will find in the source files:

1. Some partially coded tests that you will need to adapt to use your classes / data structures.
2. Methods for reading and writing data files (`.dat` files in the data folders) and image files (in `.pgm` format).
3. Additional utilities such as methods for `std::vector` arithmetic.
4. Some optimisation specific utility code that you will make use of throughout the assignment such as generating Gaussian kernels and a simple convergence function.
5. Some simplified example classes with the structure of the differentiable functions, non differentiable functions, and linear operators.

You should inspect the `CMakeLists.txt` files as well and understand the structure of the build system. This is necessary in order to understand how to add new files to the code base.

0.4 Writing In `Responses.md`

Please be sure to clearly label each question response with the section of the assignment that you are answering. Your file should be well formatted so that it is easy to read, you should write clearly, and explain your ideas as fully as you can.

1 Library Code Part 1: I/O [8 Marks]

This project will require file i/o for two main types of files: data files and image files. Data files will be used to load measurement sets ($\mathbf{y}$), as well as some information related to linear operators. Image files are used specifically for images, although the format is very similar.

The format for data files is as follows:

- The first line contains the dimensions of the image to be reconstructed separated by a space.
- The second line contains the number of data in the set.
- The third line onwards contains the data, with one item per line. A data file may use any arbitrary type for its data, but all data in a given file must be the same type. In this assignment we will consider both real (floating point or integral types) and complex (`std::complex<double>`) types.
  - A real value is just a single number.
  - A complex value is represented by two numbers separated by a space. The first value is the real part, and the second is the imaginary part. (The imaginary part is strictly numeric, there is no `i` or `j` character to denote the imaginary part.)

Image files being read are in `.pgm` format. To view them you can install a viewing extension for VSCode or use an image program such as gimp.

We have provided functions to do the following:

- Read and write well formatted data files for real and complex data.
- Read and write image files in .pgm format.

Tasks

1. Add error handling to the **data file reader**.
2. Test your data reader function.

N.B. You do not need to also add error handling to image reader, or to the write functions, but you may if you wish. (The error cases for images and data are very similar.) The image read and write functions are limited compared to the flexibility of the `.pgm` format, and only designed to read and write with specific layouts in mind, so you may need to modify if you wish to use them with images that we have not provided.

[4 marks implementation; 4 marks testing]

2 Library Code Part 2: The Generic Problem [12 marks]

2.1 Abstract Classes [4 marks]

Recall the definition of the *cost function*, which we are trying to minimise:

$C(\mathbf{x}, \Phi, \mathbf{y}) = f(\mathbf{x}, \Phi, \mathbf{y}) + g(\mathbf{x})$.

The three main components that define a given optimisation problem are the differentiable function f, the non-differentiable function g, and the linear operator Φ. In order to describe the problem in abstract terms, we don't need to know exactly *what* any of these functions actually are, we only need to know *how* they can be used. This makes them a good candidate for abstract classes.

In this section you will write an abstract class for each of these three types of function. You may use the `Quadratic`, `Identity`, and `Indicator` classes as guidance for the kind of functionality that they need to perform, but bear and in mind that these are *simple, concrete cases*, and you should be writing classes which are *abstract* and *generic*.

2.1.1 Differentiable Functions

A representation of a differentiable function f needs to be able to do two things:

- Evaluate the function for a given input, $f(\mathbf{x})$. This output is a single real number (`double`).
- Evaluate the gradient for a given input, $\nabla f(\mathbf{x})$. This output is a vector the same size and type as $\mathbf{x}$.

Tasks

1. Write an abstract class to implement a differentiable function.
   - The type of the data in the input and the gradient should be generic, and must be the same.
   - The return type of the function evaluation should be a single `double`.
   - The function evaluation should overload `operator()`.
   - The gradient should be a function called `gradient`.

2.1.2 Non-Differentiable Functions

A representation of a non-differentiable function g needs to be able to do two things:

- Evaluate the function for a given input, $g(\mathbf{x})$. This output is a single real number.
- Evaluate the proximal operator for a given input, $\text{prox}_g(\mathbf{x})$. This output is a vector the same size and type as $\mathbf{x}$.

Tasks

1. Write an abstract class to implement a non-differentiable function.
   - The type of the data in the input and the gradient should be generic, and must be the same.
   - The return type of the function evaluation should be a single `double`.
   - The function evaluation should overload `operator()`.
   - The proximal operator should be a function called `proximal`

2.1.3 Linear Operators

To implement linear operators in our code we need to represent both the linear operator itself (sometimes referred to as the *forward transform* for clarity) and its adjoint.

- The linear operator acts on input vectors $\mathbf{x}$ and produces measurement vectors $\mathbf{y}$. The types of data contained in $\mathbf{x}$ and $\mathbf{y}$ do not have to be the same. For example a

linear operator might take a vector of real numbers and return a vector of complex numbers.

- The adjoint operator acts on measurement vectors $\mathbf{y}$ and produces input vectors $\mathbf{x}$. This is used for obtaining the initial guess, amongst other things.

Tasks

1. Write an abstract class to represent a linear operator.
   - The type of data in the input and measurement vectors should be generic.
   - The forward transform, $\Phi(\mathbf{x})$, should be implemented by **overloading** `operator()`.
   - The adjoint , $\Phi^\dagger(\mathbf{y})$ should be a function called `adjoint`.

2.2 The Iterative Algorithm [5 marks]

The details of the algorithm are covered in the mathematical background section 0.1.3 of this assignment.

Tasks

You should now implement the iterative optimisation algorithm as a function.

1. This function should return the final estimate of $\mathbf{x}$ after meeting the convergence criterion.
2. The function should take the following parameters:
   - $f(\mathbf{x})$, the differentiable function
   - $g(\mathbf{x})$, the non-differentiable function
   - $\mathbf{y}$, the measurement set
   - Φ, the linear operator
   - α, the step size for the gradient
   - β, the step size of the proximal
   - $n_{\max}$, the maximum number of iterations before giving up
   - δ, the convergence threshold
   - $H(\mathbf{x}_n, \mathbf{x}_{n+1}, \delta)$, the convergence function (optional with default; see below)
3. The algorithm should terminate if $H(\mathbf{x}_n, \mathbf{x}_{n+1}, \delta)$ returns `True` or if the maximum number of iterations is reached.
4. The function should generate an initial guess by using the formula $\mathbf{x}_0 = \Phi^\dagger \mathbf{y}$ before the first iteration.
   - The result of the first iteration is therefore $\mathbf{x}_1$, and if convergence is not reached then the result returned will be $\mathbf{x}_{n_{\max}}$.
5. Your algorithm function should be programmed to work with the generic interfaces that you have already developed in the sections above.
6. It should be able to accept an arbitrary convergence function H as long as it matches the correct signature, as indicated by the provided convergence function `bool converged(vector<Tx> &x1, vector<Tx> &x2, double tol)` (see `OptimisationUtils.h`). This convergence function should be the default if no convergence function is given as an argument.
7. When deciding the types for this algorithm you should aim to **keep it as generic as possible while respecting type constraints of the problem**.

2.3 A Simple Test [3 marks]

Tasks

In this test we will check that we can solve a very simple problem: find the value x which minimises the function x^2. In this case:

- We can use $f(x) = x^2$ (using the `Quadratic` class).
- We can use $\Phi(x) = x$ (using the `Identity` class).
- We can set $g(x) = 0$ to make no contribution (using the `Empty` class).

In this case:

$\mathbf{x}_0 = \Phi^\dagger(\mathbf{y}) = \mathbf{y}$, so the value of y is your initial guess.

Each iteration of the algorithm will take a gradient step $\Delta x = 2\alpha x$.

1. Modify the `Quadratic`, `Identity`, and `Empty` classes so that they are usable with your optimisation algorithm.
2. Use the default convergence criterion.
3. The test will check that your result differs from 0 by no more than 10^{-3}.
   - How close to 0 you get will depend heavily on your convergence tolerance, so you may need to experiment to set your tolerance low enough.
   - α should be small enough that you don't immediately over-shoot your target. How you set α will affect the number of iterations your test needs.
4. In `Responses.md` describe some limitations of this as a test of our algorithm.

3 Library Code Part 2: Concrete Linear Operators [21 marks]

In order to try our code out on a meaningful imaging problem, we will create some more interesting linear operators and functions. In this section you'll implement two linear operators:

- A subsampler, which takes a sub-set of data from an image to produce a new vector with less information.
- A convolution operator, which performs a convolution. A common application of convolution in imaging is point spread which models how a single point in the real world gets rendered by a system such as a camera. One example of this would be Gaussian blur, a common model for blurring images.

3.1 Implementing a Sub-Sampling Linear Operator [7 marks]

A sub-sampling operator receives a vector of data of a given size, and returns a sub-set of that data. This linear operator can simulate data loss, or patchy data (for example incomplete detector coverage of an area).

In the following we will use the notation $\mathbf{I}$ to represent the **list of indices preserved by the subsampling process**. A sub-sampling operator is determined by the size of image it is intended to work on, and this list of indices.

Tasks

Write a sub-sampler linear operator class such that:

1. The list of preserved indices $\mathbf{I}$ should be passed into the constructor.
2. The forward transform takes an input $\mathbf{x}$ and returns a measurement $\mathbf{y}$ containing all the input values at the indices given in the constructor. For example, if the linear operator is constructed with the list of indices $\mathbf{I} = [1, 4, 2]$, then when applied to a vector $\mathbf{x} = [0, 9, 4, 1, 2, 5]$ it should return the smaller set $\mathbf{y} = \Phi(\mathbf{x}) = [9, 2, 4]$.
   - The list of indices does not need to be ordered, and the output vector should contain the elements of the input vector *in the order given in the list of indices* $\mathbf{I}$. It is important that you respect the ordering in $\mathbf{I}$ because some problems that you will run will involve reconstructing an image from a sub-sampled image and a list of indices.

3. The adjoint takes a measurement set $\mathbf{y}$ and returns an image $\mathbf{x}'$ with:
   - If $\mathbf{y} = \Phi(\mathbf{x})$ then the values in $\mathbf{x}' = \Phi^\dagger(\mathbf{y})$ should match those in $\mathbf{x}$ for all preserved indices i.e. $\mathbf{x}'_i = \mathbf{x}_i$ for all $i \in \mathbf{I}$.
   - The values at all other indices, where data has been lost, should be filled with `0`.
   - Using the same example above, where $\mathbf{I} = [1, 4, 2]$, $\mathbf{x} = [0, 9, 4, 1, 2, 5]$, and $\mathbf{y} = \Phi(\mathbf{x}) = [9, 2, 4]$, then $\mathbf{x}' = \Phi^\dagger(\mathbf{y}) = [0, 9, 4, 0, 2, 0]$.
4. You should consider error handling carefully when implementing this class.
5. Write unit tests for your sub-sampling linear operator.

3.2 Implementing a Convolution Linear Operator [14 marks]

N.B. In this section you will need to use the `FFTW` library to implement convolutions using Fourier transforms. For more information on this library and its syntax please check the `FFTW` section (0.2) at the end of the introduction.

In this example you will create a more complex linear operator. A convolution is a common operation on images which can simulate things like blurring and lens effects. Although convolutions can be calculated directly, we will calculate it in a slightly different way using a Fourier transform, as this makes it easier to define the adjoint operator. You don't need to understand exactly what a Fourier transform (or convolution) does, except for the following:

- A Fourier transform converts between two representations of a function. An image $\mathbf{x}$ can be considered a *real space function*, where the function is given a value at various *points in space*. A Fourier transform gives the *Fourier space function*, which we notate with a tilde: $FT(\mathbf{x}) = \tilde{\mathbf{x}}$. The Fourier space function represent a function as a sum of *waves*, with amplitudes defined at various *frequencies* instead of points in space. (For example, a Fourier transform would convert a sin wave in real space to a single peak in Fourier space at the frequency of that wave.)
- The Fourier transform has an inverse, which converts from Fourier space to real space: $FT^{-1}(\tilde{\mathbf{x}}) = \mathbf{x}$.
- Given an image $\mathbf{x}$ and a kernel $\mathbf{k}$, we can calculate the convolution $\mathbf{x}' = \mathbf{x} * \mathbf{k}$ by the following process:
  - Apply the Fourier transform to obtain $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{k}}$.
  - Calculate each element of $\tilde{\mathbf{x}}'$ by a pointwise multiplication: $\tilde{\mathbf{x}}'_i = \tilde{\mathbf{x}}_i \tilde{\mathbf{k}}_i$.
  - Apply the inverse Fourier transform to get $\mathbf{x}'$.

The benefit of this approach is that it makes the adjoint (the "deconvolution", which is typically extremely difficult to calculate in real space) fairly simple to define.

- Given a convolved image $\mathbf{x}'$ and a kernel $\mathbf{k}$, we can (approximately) reconstruct the original image $\mathbf{x}$ by the following process:
  - Apply the Fourier transform to obtain $\tilde{\mathbf{x}}'$ and $\tilde{\mathbf{k}}$.
  - Calculate each element of $\tilde{\mathbf{x}}$ by a pointwise division: $\tilde{\mathbf{x}}_i = \frac{\tilde{\mathbf{x}}'_i}{\tilde{\mathbf{k}}_i}$.
  - The deconvolution process can often be unstable, especially if some of the kernel values $\tilde{\mathbf{k}}_i$ are small. It is more stable to implement the following modification: $\tilde{\mathbf{x}}_i = \frac{\tilde{\mathbf{x}}'_i}{\tilde{\mathbf{k}}_i + \epsilon}$ for a small ϵ to prevent the denominator from being too small.
  - Apply the inverse Fourier transform to get $\mathbf{x}'$.

N.B. Because the convolution is implemented as a point-wise multiplication in Fourier space, **the dimensions of the image buffer and the kernel buffer must be the same**.

Tasks

Using the definitions above, write a convolution linear operator class which implements the following:

1. The constructor should take an image representing the *kernel*. The kernel must be the same size as any images which get passed to this operator.
2. The forward transform should perform a convolution. It should take a vector of real data representing an image and return another vector of real data.
3. The adjoint should be the deconvolution. It should likewise take an image and return an image.
   - You should use $\epsilon = 0.01$ by default in the pointwise division to match the example data given for testing.
   - You should take ϵ as an *optional* argument in the constructor.
4. The image input and output data must be real (`double`), but the intermediate Fourier space representations $\tilde{\mathbf{x}}'$ and $\tilde{\mathbf{k}}'$ *must* be complex (`fftw_complex*`).
   - When applying the forward transform to an image you should take the imaginary part of every pixel to be `0`.
   - After applying the backward transform you can discard the imaginary part to get a real image.
5. You should use the FFTW library to perform the forward and backward fourier transforms.
6. Test your convolution class by using the test images provided.
   - The outline of a test for this operator has been provided for you.
   - Bear in mind when comparing your images to the ones provided that the values in the image files *are rounded to the nearest integer*.

Hints

- An object of this class should be able to be applied to any supplied image *of the correct dimensions*, i.e. the source image should not be fixed at construction.
- You should try not to do unnecessary work inside the class as this operator will be called every iteration in your optimisation algorithm.
- Don't forget the scaling on the inverse transform!

4 Library Code Part 4: Concrete Functions [22 marks]

4.1 Implementing a Gaussian Likelihood function [6 marks]

In this section you will implement an non-normalised Gaussian log-likelihood function as an example of a *differentiable function*. The functional form is:

$f(\mathbf{x}, \mathbf{y}, \Phi) = (2\sigma^2)^{-1}|\Phi(\mathbf{x}) - \mathbf{y}|^2$

- This likelihood is a measure of the difference between the measurement data that we have ($\mathbf{y}$) and the measurements that we would expect from our image after applying the measurement operator ($\Phi(\mathbf{x})$).
- The parameter σ is the standard deviation of the likelihood, and it determines how sensitive our function is to variations in $\Phi(\mathbf{x})$.
- In our imaging applications the image, $\mathbf{x}$, is always a vector of real values.
- The measurements, $\mathbf{y}$, may be a vector of real values *or* a vector of complex values, depending on the measurement operator.
  - For example, if Φ is a simple sub-sampler, then $\Phi(\mathbf{x})$ and $\mathbf{y}$ are vectors of real values. But if Φ were a Fourier transform, then $\Phi(\mathbf{x})$ and $\mathbf{y}$ would be complex, because the output of a Fourier transform is complex.

Note that the calculation for the magnitude of a vector $|\mathbf{v}|^2$ is different for real numbers and complex numbers. For real numbers we have:

$|\mathbf{v}|^2 = \sum v_i^2,$

and for complex numbers we have:

$|\mathbf{v}|^2 = \sum v_i \times v_i^*,$

where v_i^* denotes complex conjugation. For a complex number $(a + ib)$, its complex conjugate is defined as:

$(a + ib)^* = (a - ib)$

N.B. The C++ standard library provides the complex type `std::complex<double>`, which has the member functions `real()`, `imag()`, and `conj()` for getting the real part, imaginary part, and complex conjugate of a complex number respectively. The operators `+`, `-`, `*`, and `/` are also overloaded for complex types.

The gradient of the log-likelihood can be calculated as follows:

$\nabla f(\mathbf{x}) = \Phi^\dagger \left(\frac{\Phi(\mathbf{x}) - \mathbf{y}}{\sigma^2} \right)$

Tasks

1. Implement the Gaussian likelihood as a sub-class of the differentiable function class.
2. The constructor should take the measurement operator, the measurements, and σ as arguments.
3. Your class should be templated to be able to use real (`double`) or complex (`std::complex<double>`) types for the measurements, depending on the type of measurements the measurement operator will produce.
4. You should test your log-likelihood function and gradient following the guidance in the test file.

4.2 Implementing an DCT L1-Norm Prior [9 marks]

Our non-differentiable function will make use of the discrete cosine transform (DCT), also provided by the FFTW library. (For more information, please see the FFTW section of the introduction.) Unlike the FT, the input and output buffers for this transform are both *real*. This transform is very common in image processing, such as jpeg compression. The DCT converts an image from a collection of pixels with different intensities to a collection of cosine-wave patterns with different intensities. The sum of all these cosine patterns gives you the image again.

For our purposes, the L1-Norm is simply a name for the **sum of absolute values of the elements in the vector**. By adding it to our cost function, we are penalising our solution for having large values in the DCT representation of the image. This kind of condition is often applied to encourage smoothness in an image.

Calculating the DCT L1-Norm involves three steps:

- Apply the DCT to the input image $\mathbf{x}$.
- Sum the absolute value of each element in $DCT(\mathbf{x})$.
- Return the sum.

Instead of a gradient, we have a *proximal operator*. In the case of the L1-Norm, the proximal operator is a *soft thresholding*. To calculate the proximal operator one must:

- Apply DCT to input image $\mathbf{x}$.

- For each element in $DCT(\mathbf{x})$, apply the soft thresholding function S with threshold τ:
  - if $\tilde{\mathbf{x}}_\mathbf{i} < \tau$ then $S(\tilde{\mathbf{x}}_\mathbf{i}) = 0$
  - otherwise $S(\tilde{\mathbf{x}}_\mathbf{i}) = \tilde{\mathbf{x}}_\mathbf{i}(1 - \frac{\tau}{|\tilde{\mathbf{x}}_\mathbf{i}|})$
- After applying the soft thresholding to the frequency space representation $\tilde{\mathbf{x}}$, apply the inverse DCT to get the modified image.
- Rescale modified image to account for unnormalised DCT.
- Return the **difference** between the original image and the modified image.

Tasks

1. Implement a DCT L1-Norm class as a non-differentiable function.
2. The constructor should optionally take the threshold parameter τ, which should default to `0.1`. In order to set up the DCT, the constructor will also need the dimensions of the image.
3. Use the FFTW library to perform the DCT and its inverse.
4. Implement both the function and the proximal operator.
5. Test your class using the test framework provided.
   - **N.B. You should use the updated `TestSphere_proximal.dat` provided if you have used the corrected FFTW interface from this version of the assignment text.**

Hints

- As with your convolution class, an object of this class should be able to be applied to any supplied image of the correct dimensions, i.e. the source image should not be fixed at construction.
- You should try not to do unnecessary work inside the class as this operator will be called every iteration in your optimisation algorithm.

4.3 Convergence Functions [3 marks]

The last part of our algorithm that we need in order to run an iteration problem is a convergence function.

We have provided one convergence function in `OptimisationUtils.h`, which looks like this:

$H(\mathbf{x}_n, \mathbf{x}_{n+1}, \delta) = \frac{|\mathbf{x}_{n+1} - \mathbf{x}_n|}{|\mathbf{x}_n|} < \delta.$

Tasks

1. You should implement the following convergence function:

   $H(\mathbf{x}_n, \mathbf{x}_{n+1}, f, g, \delta) = \frac{|C(\mathbf{x}_{n+1}) - C(\mathbf{x})|}{C(\mathbf{x})} < \delta.$
2. Explain in `Responses.md` how you can ensure this function has the correct signature to be passed into your optimisation algorithm once you have defined all the pieces of your optimisation problem. Give an explicit code example of how you would do this.

N.B. There are no marks for testing this function.

4.4 Integration Test [4 marks]

You should now perform a check that your overall algorithm is working with all the components in place. You should use the image `test/data/images/TestSphere.pgm` as the basis for your test; this will be the "true" image that you try to recover. You can simulate the measurement process by applying a measurement operator to the true image to generate a set of measurements.

Tasks

1. Write a test which checks the performance of your algorithm with your Gaussian Likelihood function, a DCT L1-Norm, and a sub-sampler which discards 20% of the data.
   - You can use the following parameters:
     - set $\sigma = 0.1$, the standard deviation of the log-likelihood.
     - set $\alpha = 0.01$, the gradient step factor.
     - set $\beta = 10^4$, the proximal step factor.
     - set $\delta = 0.4$, the convergence tolerance. (If you function converges too quickly, lower the tolerance. Don't worry too much if it doesn't converge, as long as it returns a good result at max iterations. You may need to run for up to 50-100 iterations.)
     - set $\tau = 0.1$, the soft threshold in the DCT L1-Norm.
2. You should check that reconstructed image that you get is an improvement on the dirty image ($\mathbf{x_0}$) by comparing both to the true image.

5 App Code Part 1: Simple Imaging Problems [10 marks]

In this section you will use the concrete classes you have created so far to solve two imaging problems. We have provided the *measurements* data for each of these problems, but *not* the original images. You must write an application which can reconstruct these two images, and save your results to the `images` folder.

- One data set is an image which has been **sub-sampled**.
- The other is an image which has beeen **blurred** using a convolution with a sinc kernel.

Both problems will use a Gaussian log likelihood for f and a DCT L1-Norm for g, but they require different linear operators.

Tasks

1. Write a command line application called `ImageReconstructor`. You should implement the following command line arguments:
   - `-h`: if this flag is used then the application should display a help message and then terminate; the optimisation algorithm should not be run. If additional options are specified then they should be ignored.
   - `-f`: the measurements data file
   - `-alpha`: the value of α, the gradient step size
   - `-beta`: the value of β, the proximal step size
   - `-sigma`: the value of σ in the log-likelihood function
   - `-delta`: the value of δ in the convergence function
   - `-k`: the Gaussian kernel size in pixel (one standard deviation) *only for a convolution problem*
   - `-i`: the indices file *only for a subsampling problem*
     - The `-k` and `-i` flags should be **mutually exclusive**, but the user **must** supply one.
2. If the kernel option is provided then the app should create a convolution linear operator using a Gaussian kernel of the given standard deviation.
   - You can generate a kernel of a given image size using `GenSincKernel` from `ImageUtils.h`.
3. If an indices file is provided then the app should create a sub-sampling linear operator using the provided list of indices.
   - The indices can be loaded using the `ReadData` function in `OptimisationUtils.h`.

4. If the arguments are valid then the program should do the following:
   - Calculate the dirty image ($\Phi^\dagger(\mathbf{y})$) and save it to a file. This should be named the same as the measurement input file name with `_dirty` appended before the file extension. (The file extension should be `.pgm`.)
   - Run the optimisation problem using the appropriate differentiable function, non differentiable function, linear operator, and parameters.
     - $\sigma = 1.0$
     - $\alpha = 1.0$
     - $\beta = 10^4$
     - $\tau = 0.1$
     - max steps = 500
   - Save the reconstructed image to a file. This should be named the same as the measurement input file name with `_reconstructed` appended before the file extension. (The file extension should be `.pgm`.)
   - Your solution may not converge before reaching the maximum number of iterations. This is not necessarily a problem and your optimisation algorithm should still return a result which you can use.
5. Report all the command line parameters used for each run in your `Responses.md` file. Run your app for the following two problems:
   - **A sub sampling problem** using:
     - the measurements file `data/measurements/UtahTeapot_subSampledMeasurements.dat`
     - the indices file `data/operators/UtahTeapot_sampledPixels.dat`.
   - **A convolution problem** using:
     - the measurements file `data/measurements/UtahTeapot_convolved.dat`
     - a sinc kernel with scale 1.0.
6. **Make sure that you place your dirty and reconstructed images into the `images` folder before you submit.**

Hints

- You can use basic C++ command line parsing: http://www.cplusplus.com/articles/DEN36Up4/ or you could try integrating a command line parser such as: https://github.com/CLIUtils/CLI11 (which is header only). If you want to use `CLI11` you should include it as a sub-folder inside your submission and make sure that you modify your cmake so that your project builds correctly as the markers will not necessarily have this library on their own machine.
- You should compile with optimisation flags `-O2` or `-O3` turned on in your cmake and debugging turned off (no `-g` flag) to speed up running these problems.
- The quality of reconstruction can be sensitive to parameters, so you will not be marked on the quality of your final images as long as your code is functioning properly.
- The sub-sampling problem should show a clear improvement. The blurring problem may have a final result similar to the dirty image depending on how small the value of ϵ is and how accurate the deconvolution is. You should see a clear improvement if $\epsilon = 0.1$.
- The parameters suggested will not necessarily give the *best* results, but should work without the solution diverging.

6 Library Code Part 4: Further Operators and Abstract Operator Composition [12 marks]

In this final part of our library development, we will make our linear operator code more powerful allowing new linear operators to be defined as the *composition* of two linear operators. We will

also introduce one final linear operator, the Fourier transform operator.

6.1 Fourier Transform Operator [3 marks]

Write code using the FFTW library to implement the Fourier transform and its inverse as a linear operator.

1. The forward transform is the forward FT; it should take an image (i.e. real vector) as input and return a complex vector.
2. The adjoint is the inverse FT[4], including rescaling (as discussed in the FFTW section of the introduction). It should take a complex vector as input and return an image.
3. The constructor should take the intended image dimensions as arguments.
4. Since the FT is invertible, implement a unit test to check that $\Phi^\dagger(\Phi(\mathbf{x})) \approx \mathbf{x}$.

Hints

- Use FFTW to implement to handle the transforms. Remember to use the FT and not the DCT!
- You can discard the complex part after the inverse FT and just take the real values.

6.2 Operator Composition [9 marks]

New linear operators can be defined as the **composition** of two linear operators $\Phi \circ \Psi$. The composition is defined as:

$(\Phi \circ \Psi)(\mathbf{x}) = \Psi(\Phi(\mathbf{x}))$,

i.e. it is the application of one linear operator after the other. The adjoint operator is defined as $(\Phi \circ \Psi)^\dagger = \Psi^\dagger \circ \Phi^\dagger$ (note that the order of the adjoint operators is reversed compared to the forward operator). When applied to a measurement set, the adjoint operator looks like this:

$(\Phi \circ \Psi)^\dagger(\mathbf{y}) = \Phi^\dagger(\Psi^\dagger(\mathbf{y}))$

i.e. the application of one adjoint operator after the other, in the reverse order. This makes sense if we consider the process of doing the forward operation followed by the adjoint operation on an image $\mathbf{x}$:

$x \approx (\Phi \circ \Psi)^\dagger((\Phi \circ \Psi)(\mathbf{x})) = \Phi^\dagger(\Psi^\dagger(\Psi(\Phi(\mathbf{x})))$

Tasks

1. Write code to implement the composition of linear operators.
2. You may implement this operator composition as you see fit.
3. Bear in mind the input and output types of each operator, and the input and output type of the result of the composition.
4. The result of the composition must derive in some way from your abstract linear operator class in order to be usable in the rest of the code.
5. Describe in `Responses.md` your implementation choices.
    - Explain how the type correctness is ensured whilst keeping the code as generic as you can.
    - Explain any choices with regards to data copying, sharing, and ownership that are relevant to your implementation.
6. Test your composition operators behave as expected.

[4] The FT (when normalised) is a *Unitary* transform, which means that it's adjoint is its inverse. This is the only linear operator that we've used so far where this is the case!

- Using the `Identity` operator, test the property $\Phi \circ \mathbf{Id} = \mathbf{Id} \circ \Phi = \Phi$, since the identity doesn't change the input that it is given.

7 App Code Part 2: A More Complex Imaging Problem [5 marks]

Write a second application called `FrequencyReconstructor`.

1. Using your operator composition define a new operator which implements the following measurement process:
   - The image is converted to Fourier space using the a Fourier transform. (This represents a measurement taken by gathering frequency information rather than spatial information.)
   - The Fourier space representation is then sub-sampled. (This represents incomplete frequency coverage.)
   - **The output of this overal process will be a complex vector of a smaller size than the image.**
2. You can use the same command line arguments as before, except you no longer need the `-k` option as we will not be using convolution in this application. The `-i` flag is now mandatory.
3. When running the application you should use:
   - the data file `data/measurements/UtahTeapot_frequencyLossMeasurements.dat` for your measurement set $\mathbf{y}$. These are complex measurements in Fourier space.
   - the indices file `data/operators/UtahTeapot_sampledFrequencies.dat` to get the indices required for the sub-sampler to succesfully reconstruct the image. These are the indices of all the Fourier space frequencies which have been retained for this image.
4. Apply this to the provided measurements file to reconstruct the image, using the following parameters as a guide:
   - Set $\sigma = 1.0$ in the Gaussian log-likelihood
   - set $\alpha = 1.0$
   - set $\beta = 10^4$
   - set $\tau = 0.1$
5. Save the dirty image and your final reconstruction; you should use the same naming conventions as the previous application. You should make sure that these have been saved to the `images` folder before you submit.

As before, you will be marked on the quality and correctness of your code rather than the quality of your reconstructed image.

8 Additional Considerations (10 marks)

8.1 Good practices

We expect you to engage with typical good practice shown throughout the course. Specifically you will be marked on:

- Appropriate naming for functions and classes.
- Good code organisation (i.e. appropriate choice of files and folders).
- Informative and useful git commit log.
- Effective in-code commenting.
- Code formatting.

Hints:

- See section 9 (Additional Hints)
- Familiarise yourself with writing good git commit messages.
- Familiarise yourself with Best practices for writing code comments.
- You can use formatting tools like `clang-format` from inside VS Code to help format your code.

8.2 User instructions

Update the front page `README.md` to give clear build instructions (including what is required to be able to build), an explanation of what the repository is for, and clear instructions (with examples) for any command line applications.

9 Additional Hints

This section contains only extra, optional hints; there are no specific instructions or associated marks.

9.1 Code Layout

- Make sure all header (`.h`) files have an include guard to prevent accidental double declaration of variables/functions/classes. Alternatively you can use`#pragma once`; it's less typing but it is not part of standard so not guaranteed to be available for every compiler.
- Header files should only contain function/class/variable declarations (the interface) not their definitions (the implementation) *except* in the case of class/function templates where implicit instantiation is *useful*. Definitions should go in a corresponding source (`.cpp`) file; this keeps interface and implementation separate, speeds up compilation and avoids double definitions of functions/classes/variables.
- Choose a consistent formatting style (for example the google C++ style guide) and stick to it. The focus should be on readability.
- Use descriptive variable, function and class names that describe the purpose/intent of an object and would be understandable to someone reading the code for the first time.
- Be consistent with indentation. Use 2, 4, etc spaces and stick to it. Avoid tabs as these can show up differently on different editors. You can use automatic tooling to help you with that.
- Comments in the header files should describe what a class/function is for and how it should be used (unless it is obvious from the function name).
- When commenting in general, avoid stating the obvious or describing literally what the code does when commenting in the implementation (`.cpp`) source files. Use comments for *tricky, non-obvious, interesting, or important parts of your code* (from cppguide#Comments).

9.2 Unit Tests

- Each unit test `TEST_CASE` should at least one meaningful assertion to check a condition has been met. Avoid meaningless checks such as `REQUIRE(true)` that will always pass. To check that a class can be instantiated you can use `REQUIRE_NOTHROW` or `REQUIRE_THROWS` to check that an exception is not/is thrown as expected, for example:

```
TEST_CASE( "FileReader file reader instantiation" , "[FileReader]")
{
```

```
  REQUIRE_NOTHROW(FileReader("file_that_exists.txt"));
  // should throw an exception for non-existent file
  REQUIRE_THROWS(FileReader("file_that_does_not_exist.txt"));
}
```

- To improve coverage of your unit tests consider whether you have fully tested the behaviour of the object or function that you are unit testing:
  - Have you tested for known solutions that you can compare to?
  - Have you tested a range of representative use cases, not just one specific case?
  - Have you tested for failure (like invalid inputs where you expect the constructor to throw an exception) and edge cases (special cases like the square root of 0)?
- Use approximate testing for floating points values in almost all cases, except for very specific cases where the *exact* value is known such as checking instantiation to `0`.

9.3 Commit Log

- Commit messages should describe the changes being made and have enough information that someone could come back and find out where a specific change was introduced
- Avoid bundling many changes into a single commit: this makes it harder to understand what has been changed in each commit and also harder to use tools such as `git bisect` to locate a change that may have introduced a bug
- Keep the first line of commit message concise (i.e. brief but with lots of info) and add more details as part of the message body if needed